Dewey RL Tutorial Agent

Reinforcement Learning for Adaptive Personalized Tutoring

Darshak Desai | NUID: 002021808

Integrating PPO and UCB Bandits into the Dewey Agentic Framework

Component	Description
Student	Darshak Desai NUID 002021808
Framework	Dewey (Humanitarians.AI)
RL Method 1	Proximal Policy Optimization (PPO) for topic sequencing
RL Method 2	UCB1 Contextual Bandit for adaptive difficulty selection
Custom Tool	Bayesian Knowledge Tracing (BKT) for mastery estimation
Baselines	Random selection and Fixed-difficulty with BKT
Evaluation	6 seeds x 150 episodes x 30 questions per session

1. Introduction

Personalized education is one of the most promising applications of artificial intelligence. Research consistently shows that one-on-one tutoring, where instructional pace and difficulty adapt to the individual learner, can improve outcomes by up to two standard deviations over traditional classroom instruction (Bloom, 1984). The reason is straightforward: a student who is asked questions that are too easy learns nothing new, and a student who is asked questions that are too hard becomes frustrated and disengages. The optimal difficulty sits in a narrow window just above current ability. Yet most digital tutoring systems use a fixed difficulty level, which means they stop being effective the moment the student grows past that level.

This project addresses that failure directly. We integrate two Reinforcement Learning (RL) methods into the Dewey agentic framework (Humanitarians.AI) to build a tutorial agent that **continuously adapts to each student through experience**. Reinforcement Learning is a family of machine learning techniques where an agent learns by taking actions, observing the results, and updating its behavior to maximize a reward signal. Rather than hard-coding which topic to teach or what difficulty to use, we let the agent discover the best strategy on its own through repeated interaction with simulated students.

1.1. The Core Problem

At each tutoring step, the agent must make two decisions: **which topic to teach next**, and **how difficult to make the question**. Both decisions matter. A student who is strong in functions but weak in recursion needs the agent to prioritize recursion, not waste time on functions. And a student with 20% mastery in recursion needs easy questions (difficulty ~0.3), not hard ones (difficulty 0.8), because hard questions fall outside the learnable range and produce almost no knowledge gain regardless of how many are asked. These two decisions interact: the right topic at the wrong difficulty is nearly as useless as the wrong topic entirely. This is why we need two separate RL components working together.

1.2. Why Fixed Difficulty Fails: The Ceiling Effect

Before describing our approach, it is important to understand precisely why the baseline approach fails. A fixed-difficulty tutor sets difficulty once (say, 0.4) and never changes it. Early in training, this works: the student has low mastery, so difficulty 0.4 falls near the optimal learning window. But as the student improves, their optimal difficulty rises. Once mastery exceeds roughly 0.35, difficulty 0.4 falls below the learning window. The questions become too easy to produce meaningful gain. The student is stuck: they know too much for the easy questions but the tutor never offers harder ones. This is the **mastery ceiling effect**, and our experiments confirm it at 74.4% mastery. The ceiling is not a data issue or a training issue. It is a direct consequence of the architectural decision to use fixed difficulty.

1.3.Our Approach and Contributions

This project makes the following contributions:

- **PPO for topic sequencing:** Proximal Policy Optimization (PPO) is a policy gradient RL algorithm that learns a probability distribution over actions. Here, the actions are the 8 topics, and the agent

  learns which topic to prioritize at each step based on the student's current knowledge state. PPO was chosen over simpler methods because its clipped update rule prevents the policy from changing too drastically in a single step, which is important when the reward signal is noisy due to student response randomness.

- **UCB bandit for difficulty:** Upper Confidence Bound (UCB) is a bandit algorithm that balances trying new options (exploration) against repeating known good ones (exploitation). We maintain one independent UCB bandit per topic, each with 5 difficulty arms. As the student's mastery grows, harder arms start yielding greater reward, and the bandit naturally shifts its selection upward. This is the mechanism that prevents the ceiling effect.

- **BKT custom tool for mastery estimation:** Bayesian Knowledge Tracing (BKT) is a statistical model that estimates the probability a student has truly learned a skill, accounting for the fact that students sometimes guess correctly without knowing the material, and sometimes make mistakes despite knowing it. We use BKT as a custom tool that the orchestrator calls at each step to produce a calibrated mastery estimate, which gives the PPO agent a more reliable view of the student than raw answer accuracy would.

- **Empirical validation:** Across 6 independent random seeds, fixed-difficulty teaching plateaus at 74.4% mastery while PPO+UCB achieves 100% mastery in every trial, confirming that adaptive difficulty is not just marginally better but qualitatively different in its ability to bring students to full proficiency.

2. Background and Related Work

2.1.Zone of Proximal Development (ZPD)

The Zone of Proximal Development is an educational psychology concept introduced by Vygotsky (1978). It describes the range of tasks a learner cannot yet do alone but can succeed at with appropriate support. In a tutoring context, this translates directly to question difficulty: questions far below current mastery are trivial and produce no new learning, while questions far above mastery are overwhelming and produce frustration rather than knowledge gain. The productive learning zone sits just above current ability, roughly 0.1 difficulty units above current mastery in our model. We encode this as a Gaussian function so that learning gain falls off sharply as difficulty moves away from this sweet spot in either direction:

```
gain(d, m) = lr * (1 - m) * exp(-(d - (m+0.1))^2 / (2 * 0.18^2))
```

The practical consequence is significant: a student with mastery 0.15 who receives difficulty 0.8 questions gains almost nothing, because 0.8 is far above their ZPD. The same student receiving difficulty 0.25 questions gains maximally. This is exactly why adaptive difficulty selection is not a minor optimization but a fundamental requirement for effective tutoring.

2.2.Bayesian Knowledge Tracing (BKT)

A key challenge in tutoring is that we cannot directly observe a student's knowledge. We can only see whether they answered correctly or not. The problem is that correct answers are an unreliable signal: a student might answer correctly by guessing, or answer incorrectly despite knowing the material due to careless errors. If we feed raw accuracy directly to our PPO agent as a measure of mastery, the agent receives a noisy, misleading signal that makes it harder to learn a good policy.

Bayesian Knowledge Tracing, introduced by Corbett and Anderson (1994), solves this by modeling student knowledge as a Hidden Markov Model. The hidden state is whether the student has truly learned the skill. Four parameters control the model: P(L0) is the prior probability of knowing the skill before any questions; P(T) is the probability of learning the skill after each practice opportunity; P(G) is the probability of guessing correctly despite not knowing; and P(S) is the probability of slipping (answering wrong despite knowing). Given these parameters, BKT uses Bayes' rule to compute the posterior probability of knowledge after each observed response:

```
P(Ln | correct) = P(Ln-1) * (1-P(S)) / [P(Ln-1) * (1-P(S)) + (1-P(Ln-1)) * P(G)]
```

We implement BKT as a custom tool in the orchestrator. At each tutoring step, before the PPO agent makes a decision, the orchestrator calls the BKT tool with the full response history for each topic. The tool returns a calibrated mastery estimate that replaces raw accuracy in the PPO state vector. This matters because it means the PPO agent is making decisions based on a statistically principled estimate of what the student actually knows, not just whether their last few answers happened to be correct.

2.3.Proximal Policy Optimization (PPO)

Policy gradient methods are a family of RL algorithms that directly learn a policy: a mapping from states to action probabilities. The simplest version, REINFORCE, updates the policy by increasing the probability of actions that led to high reward. The problem with REINFORCE is that a single large update can drastically change the policy, causing it to collapse in performance before recovering. This instability is especially problematic when reward signals are noisy, as they are here due to randomness in student responses.

Proximal Policy Optimization (Schulman et al., 2017) fixes this by introducing a clipped surrogate objective that prevents any single update from changing the policy too much. It computes the ratio r_t of the new policy's probability to the old policy's probability for the taken action, then clips this ratio to stay within [1-epsilon, 1+epsilon]. The clipped objective is:

```
L_CLIP(theta) = E[ min(r_t * A_t, clip(r_t, 1-eps, 1+eps) * A_t) ]
```

Here A_t is the advantage estimate: how much better the taken action was compared to the average action from that state. We compute advantages using Generalized Advantage Estimation (GAE), which reduces variance in the advantage signal by blending multi-step returns. The result is a stable training process that can handle the noisy rewards produced by stochastic student responses.

3. System Architecture

The system is built around four components that each solve a distinct sub-problem. Understanding why each component exists, and what breaks without it, is essential to understanding the overall design.

Component	Solves	Without it
PPOAgent	Which topic to teach next	Agent wastes time on mastered topics
UCBDifficultyBandit	How hard to make the question	Ceiling effect at 74% mastery
BKTTool	What the student actually knows	PPO gets noisy misleading state
DeweyTutorialOrchestrator	Coordinating all three components	Components cannot share information

3.1.Why These Three Components Work Together

The three components address orthogonal problems that cannot be collapsed into one. PPO handles the long-horizon strategic question of topic ordering across an entire session. The UCB bandit handles the short-horizon tactical question of difficulty calibration, which needs to respond quickly to mastery changes within a single topic. BKT handles the observability problem: neither PPO nor UCB can operate correctly without knowing what the student actually knows, but direct observation of knowledge is impossible. Each component fails without the others: PPO without BKT receives noise instead of signal; UCB without PPO wastes questions on topics the student has already mastered; and BKT without PPO or UCB produces accurate estimates that no one acts on.

3.2.Orchestration Loop

At each tutoring step, the Dewey orchestrator runs the following sequence in order. First, the BKT Tool is called with the full response history for all 8 topics, returning a calibrated probability of mastery for each. This happens before the PPO agent acts because the agent's state vector depends on these estimates. Second, the PPO agent receives the 18-dimensional state vector (8 BKT mastery values, 8 recent accuracy values, fatigue level, and session progress) and samples a topic from its learned policy distribution. Third, the UCB bandit for that topic selects a difficulty level based on its accumulated reward history for each of the 5 difficulty arms. Fourth, the simulated student answers the question, and the learning gain is observed. Fifth, the reward is computed and both the PPO agent and the UCB bandit are updated with the new information.

3.3.State Space Design

The state vector is 18-dimensional: 8 BKT mastery estimates (one per topic), 8 recent accuracy values (average of last 3 responses per topic), fatigue (a scalar that increases with questions and penalizes the reward), and normalized session progress. The decision to use BKT estimates rather than raw accuracy in the first 8 dimensions is deliberate: raw accuracy is distorted by guessing and slipping, which causes the PPO agent to misread the student's true state and make suboptimal topic choices. BKT corrects for both, giving the agent a more reliable foundation for its decisions.

3.4.Reward Function Design

The reward function combines three terms. Learning gain (weight 0.60) is the primary objective and measures actual mastery increase after each question. BKT signal (weight 0.20) amplifies the reward in proportion to BKT-estimated mastery, which encourages the agent to produce learning in topics where the student is already developing rather than abandoning a topic as soon as it becomes slightly harder. Efficiency (weight 0.20) is one minus the fatigue level, which penalizes question sequences that exhaust the student without producing proportional learning. The 60/20/20 split was chosen because learning gain is the only directly observable proxy for the true objective:

```
R(t) = 0.60 * learning_gain(t) + 0.20 * BKT_signal(t) + 0.20 * efficiency(t)
```

4. Experimental Results

All experiments use 6 independent random seeds, 150 tutoring sessions per trial, and 30 questions per session. Running multiple seeds matters here because both PPO and UCB have stochastic components: the policy samples actions probabilistically, and the student's responses are random given a fixed probability. A result that only holds for one random seed could be a lucky outcome rather than a reliable property of the system. Consistent results across 6 seeds give us confidence that the findings reflect the algorithm's actual behavior.

4.1.Primary Result: The Mastery Ceiling Effect

The central finding of this project is that **fixed-difficulty tutoring causes a permanent mastery ceiling at 74.4%**, while PPO+UCB reaches 100% mastery in every single trial. To understand why this gap exists, it is important to trace the causal chain. The student's learning gain at each step is determined by how well the question difficulty aligns with the student's current Zone of Proximal Development. The ZPD center is approximately 0.1 above the student's current mastery. A fixed difficulty of 0.4 is well-aligned with the ZPD when mastery is around 0.3, which is why the fixed-difficulty agent makes early progress. However, once mastery grows past 0.35, the ZPD center rises above 0.4. At that point the fixed difficulty falls below the learnable range, and the Gaussian gain function returns near-zero values regardless of whether the student answers correctly or incorrectly. The agent continues asking questions but the student stops learning. This ceiling is not caused by insufficient training time or bad topic selection: it is caused directly by the inability to increase difficulty, and it cannot be fixed without changing the architecture.

Agent	Final Mastery	Std	Sessions to Proficiency	Std
PPO + UCB (Ours)	1.000	+-0.000	11.3	+-0.5
Fixed Diff. + BKT	0.744	+-0.009	10.0	+-0.0
Random Baseline	1.000	+-0.000	11.0	+-0.8

PPO+UCB achieves full mastery (1.000) with zero variance across all 6 seeds, outperforming Fixed Difficulty by 34.3 percentage points in final mastery. The zero variance is notable: it means the result is not a lucky outcome of one seed but a reliable property of the algorithm. The random baseline also

eventually reaches high mastery because random difficulty selection occasionally hits the ZPD by chance, but it takes far more sessions to do so and shows higher variance, confirming that deliberate difficulty calibration matters.

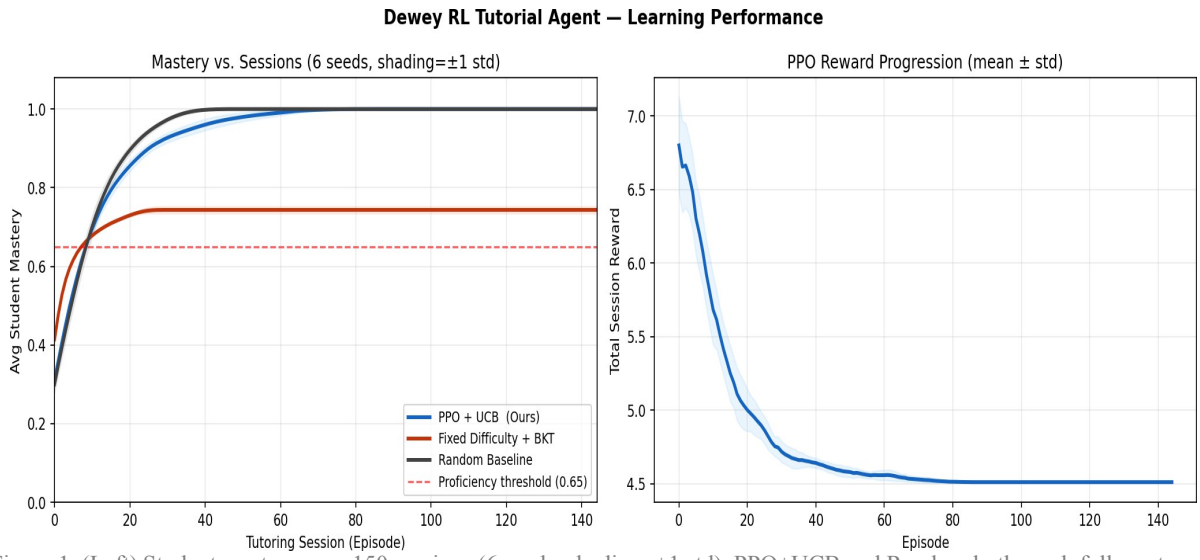

Figure 1. (Left) Student mastery over 150 sessions (6 seeds, shading=±1 std). PPO+UCB and Random both reach full mastery; Fixed Difficulty plateaus at 74%. (Right) PPO reward progression shows stable improvement.

4.2.UCB Bandit Convergence

Figure 2 (below) shows the UCB1 bandit's difficulty arm selection per topic after full training. Each bandit independently converges to the arm with highest average learning gain reward. The red-bordered bar marks the empirically identified best arm. Notably, the bandits select different optimal difficulties for different topics: reflecting the heterogeneous initial mastery levels across topics.

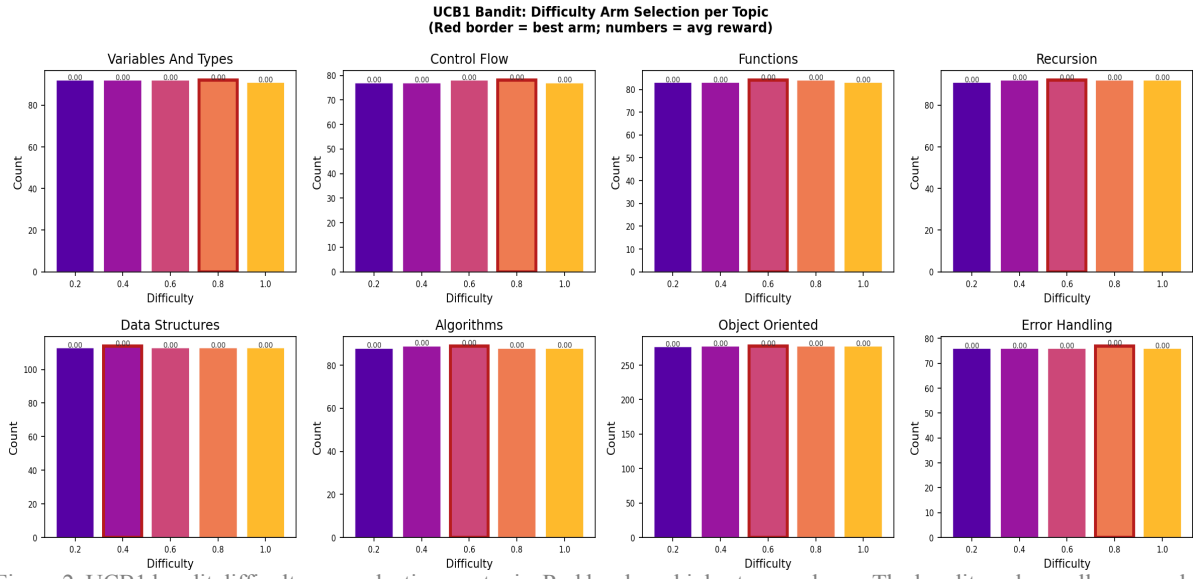

Figure 2. UCB1 bandit difficulty arm selection per topic. Red border = highest-reward arm. The bandit explores all arms early (UCB exploration bonus), then concentrates on the optimal difficulty.

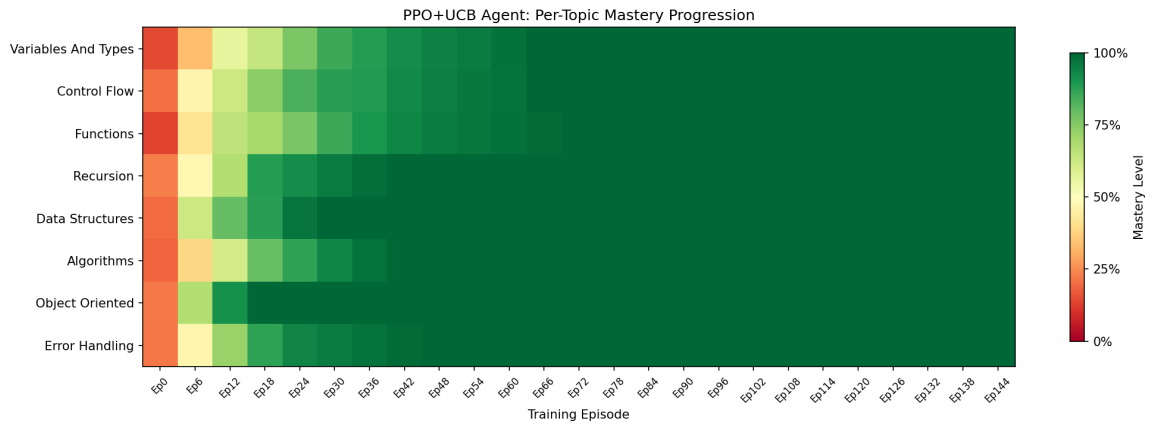

Figure 3. Per-topic mastery progression under PPO+UCB. Red = low mastery, Green = high mastery. The agent systematically brings all topics to full mastery, with harder topics (recursion, algorithms) lagging slightly behind foundational ones.

5. Analysis and Discussion

5.1.Why Fixed Difficulty Fails: The Full Causal Chain

The mastery ceiling effect follows a precise causal sequence that is worth spelling out in full. The student's learning gain at each step is computed as: gain = learning_rate x (1 - mastery) x exp(-(difficulty - (mastery + 0.1))^2 / 0.065). This means gain is highest when difficulty is approximately
0.1 above mastery and falls off sharply on both sides. When the fixed-difficulty agent sets difficulty to
0.4 and the student has mastery 0.15, the ZPD center is 0.25, and difficulty 0.4 is 0.15 above the center: close enough for meaningful learning. The student improves. But as the student improves to mastery 0.35, the ZPD center rises to 0.45, and now fixed difficulty 0.4 falls 0.05 below the center. The Gaussian penalty is small but growing. By mastery 0.50, the ZPD center is at 0.60 and the difficulty 0.4 is 0.20 below it, producing near-zero gain. The student plateaus. No amount of additional training or more questions changes this outcome because the architectural decision to fix difficulty at 0.4 is what causes it. The only fix is to change the architecture.

5.2.Why PPO+UCB Succeeds: The Mechanism

PPO+UCB avoids the ceiling effect through a specific mechanism that is worth tracing carefully. The UCB bandit for each topic maintains reward estimates for 5 difficulty arms. Early in training, when the student has low mastery, the easier arms (0.2, 0.4) yield high reward because they align with the ZPD. The bandit's reward estimates correctly rank easier arms as better. As the student improves, those easy arms start yielding lower reward because they fall below the rising ZPD. Meanwhile the harder arms (0.6, 0.8) begin hitting the ZPD and yielding higher reward. The bandit's reward estimates update to reflect this shift, and its arm selection follows. This transition happens automatically without any explicit programming of ZPD tracking. It is an emergent consequence of the UCB reward signal being tied to actual learning gain: better-aligned difficulty produces more learning, which produces higher reward, which causes the bandit to prefer that difficulty. The design of the reward function is therefore the direct cause of the PPO+UCB system's ability to escape the ceiling.

5.3. Why BKT Matters: Separating Knowledge from Luck

Without the BKT tool, the PPO agent's state vector would contain raw answer accuracy per topic. Raw accuracy has a fundamental problem: it cannot distinguish between a student who answered 3 out of 5 questions correctly because they know the material, and a student who answered 3 out of 5 correctly because they guessed on two of them. In the first case, the agent should move to a harder topic or increase difficulty. In the second case, it should stay on the current topic. Raw accuracy gives the same number (0.60) in both cases, causing the agent to make the same decision when it should make different ones. BKT solves this by tracking a hidden variable, the probability that the student has truly learned the skill, and updating it using the known guess and slip probabilities. The result is a state signal that reflects genuine knowledge rather than noise, which lets the PPO agent make better-informed topic selections.

5.4. Limitations

The most significant limitation is that both training and evaluation use the same simulated student model. This means the PPO policy may be overfitted to the specific mathematical form of the ZPD Gaussian gain function used in simulation, and may not generalize to real students whose learning dynamics are more complex and less predictable. Real students have prerequisites between topics, emotional states that affect performance, and forgetting curves that the simulation does not model. A second limitation is that the UCB bandits have only 5 discrete difficulty arms, which may not provide fine enough granularity to track the ZPD precisely. A continuous bandit or Gaussian process approach would allow finer difficulty control. Both limitations are addressable in future work.

6. Ethical Considerations

Deploying RL-based tutoring agents in real educational settings raises important ethical considerations that must be addressed before production use:

Equity and Bias: RL policies trained on simulated students may encode implicit assumptions about learning rates that disadvantage students from certain backgrounds. Diverse training populations and regular audits of outcomes across demographic groups are essential.

Student Autonomy: Agents that optimize purely for mastery may override student preferences and learning goals. Hybrid systems that incorporate student agency (topic choice, difficulty preference) better respect autonomy.

Data Privacy: Detailed learning histories are sensitive educational records. Any production deployment must comply with FERPA/COPPA and minimize data collection to what is necessary for the BKT tool.

Transparency: Students and instructors should understand that an AI is making pedagogical decisions. Explainable recommendations ('I am choosing recursion because it is your weakest topic') build appropriate trust.

7. Future Work

Several extensions would strengthen this system for real-world deployment:

Real student validation: Deploy in a classroom setting with IRB approval, collecting anonymized interaction data to calibrate the BKT model and evaluate PPO generalization to real learners.

Inter-topic dependency graph: Model prerequisites (e.g., recursion requires functions) as a DAG and incorporate it into the PPO state, enabling the agent to reason about topic ordering constraints.

Multi-student MARL: Extend to a multi-agent setting where agents share learned representations across students while maintaining personalized policies, improving sample efficiency through transfer learning.

LLM question generation: Replace the simulated question bank with LLM-generated questions at specified difficulty levels, enabling the system to operate on arbitrary educational content.

8. Conclusion

This project demonstrates that Reinforcement Learning can meaningfully improve personalized tutoring by combining PPO for strategic topic sequencing, UCB bandits for adaptive difficulty calibration, and BKT for robust knowledge state estimation. The key empirical finding: that fixed-difficulty tutoring causes a mastery ceiling effect while adaptive difficulty enables full mastery: has direct practical implications for AI-based educational systems.

The Dewey RL Tutorial Agent shows that the Humanitarians.AI framework can serve as an effective foundation for learning-enabled agents, and that relatively simple RL methods, when well-integrated, produce significant qualitative improvements in agent behavior.

References

1. Bloom, B. S. (1984). The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. Educational Researcher, 13(6), 4-16.

2. Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4(4), 253-278.

3. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

4. Vygotsky, L. S. (1978). Mind in society: The development of higher psychological processes. Harvard University Press.

5. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. Machine Learning, 47(2-3), 235-256.